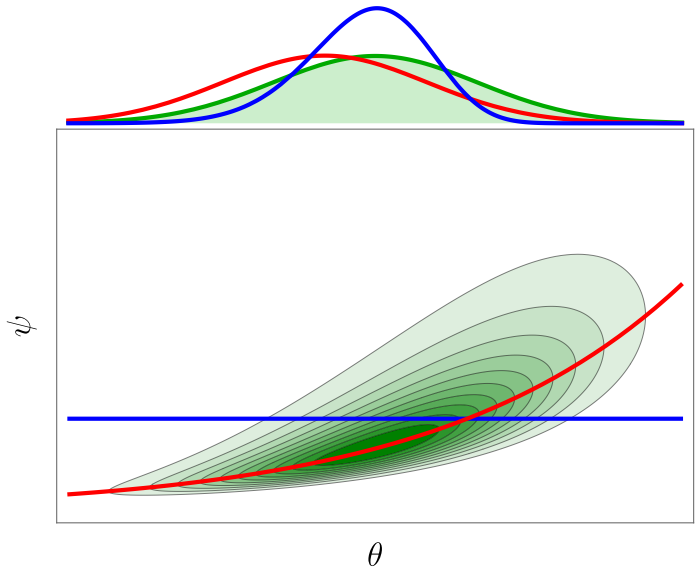


CMP++
Technical Documentation

Version 2.0

Omar Kahol

29 September 2025

Contents

List of Code Listings

1 Introduction

This document is the technical documentation for the library CMP++[1], a library primarily built for performing Bayesian calibration of computer models using several methods:

1. Full Bayes.
2. Adaptive approaches (like CMP and FMP) .
3. Sequential approaches (like KOH).

The library also implements several Uncertainty Quantification (UQ) tools such as Gaussian Processes (GPs), Markov Chain Monte Carlo (MCMC) methods, Polynomial Chaos Expansions (PCE), Sobol sensitivity analysis and much more. The library is written entirely in C++ and currently provides no high level configuration file hence the pre-processing should be done using a C++ application. The post-processing can be performed using any library or software capable of handling .csv input files.

The library can be compiled as a static library or dynamic library and depends on the following libraries:

1. Eigen 3.4.0[2] for linear algebra.
2. NLopt 2.7.1[3] for gradient-free and gradient-based optimization.
3. LIBSVM[4] for support vector machines.
4. Doxygen[5] to generate the documentation.

These libraries should be installed in your system before proceeding. A detailed installation guide is provided in the next section. Furthermore, the test cases provided in the library use Matplotlib++ for plotting the results. Though this library is not required for the compilation of CMP++ it is strongly recommended to install it if you want to run the test cases. If you choose not to install it, you will have to modify the test cases accordingly and save the outputs as CVS files to be post-processed using another software.

[1] https://github.com/omarkahol/cmp.git
[2] https://eigen.tuxfamily.org/index.php?title=Main_Page
[3] https://nlopt.readthedocs.io/en/latest/
[4] https://www.csie.ntu.edu.tw/~cjlin/libsvm/
[5] https://www.doxygen.nl

2 Installation

This section is dedicated to the installation of the CMP++ library and its dependencies. Expert users can skip this section, install the dependencies themselves, compile CMP++ using the provided Makefile and link it to their target application either as a static or dynamic library. The following workflow is, however, strongly recommended.

First and foremost, make sure that you have a C++ compiler and cmake installed in your system. The latter can be checked by calling the following command in the terminal,

```
make --version
```

The output should indicate the version installed. If this is the case, you can proceed to install the dependencies.

2.1 Installing Eigen

Eigen is a header only library so you just need to download the latest version from the official website[6] and copy the folder into a known location. In this guide, I will assume that you have copied it into the folder HOME/opt/eigen-3.4.0 but you can choose whatever location you prefer. To use Eigen in your code, simply include the main header file

```
1 #include <Eigen/Dense>
```

2.2 Installing NLopt

NLopt is compiled and linked as a dynamic library. This means that we will compile it as .dylib (in MacOs) file and we'll link it to our code during compilation. To install NLopt, you should download the latest version from the official website[7].

[6] https://eigen.tuxfamily.org/index.php?title=Main_Page
[7] https://nlopt.readthedocs.io/en/latest/

To install NLopt in HOME/opt copy the downloaded file in that folder and make sure that the name is something like nlopt-2.7.1 (otherwise simply rename the folder), access it via the terminal and type the following commands.

```
mkdir build
cd build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/opt/nlopt-2.7.1 ..
make
make install
```

This will compile the library into HOME/opt/nlopt-2.7.1 but you can change that by substituting HOME/opt in line 3 with whatever folder you desire. If the installation was successful, you should see that, in the nlopt folder you have an include folder which contains some **header files** and a lib folder which should contain (in MacOs) a **libnlopt.dylib** file. These two folders and files are the only one required so the rest can be safely removed.

Now that we have compiled the library another operation must be performed. It is indeed necessary to append the lib folder path to the system path so that the executable knows where to look for the library during runtime. In MacOs, it is sufficient to add the following line to the .zshrc file in the home folder and restart the terminal

```
export DYLD_LIBRARY_PATH=${DYLD_LIBRARY_PATH}:$HOME/opt/nlopt-2.7.
↪  1/lib
```

If this is not properly done, your code will compile and link properly otherwise you will see an *unresolved external symbol* error message at runtime.

2.3 Installing LIBSVM

LIBSVM is another library which is compiled and linked as a dynamic library. To install it, you should download the latest version from the official website[8]. After downloading, compilation and linking is performed in a similar way as for NLopt.

[8] https://www.csie.ntu.edu.tw/~cjlin/libsvm/

2.4 Installing CMP++

In this subsection, I will show how to compile CMP++ as a static and dynamic library. First download the code the official GitHub repository and copy it into a known location. In this guide, I will assume that you have copied it into the folder HOME/opt/CMP++ but you can choose whatever location you prefer. To proceed, open the makefile in the base folder and modify the following lines according to your configuration,

```
# Define the C++ compiler to use
CXX = g++-15

# Define any compile-time flags
CXXFLAGS := -std=c++20 -O3 -fopenmp -mmacosx-version-min=14.0
 ↪  -DNDEBUG
DEBUG_CXXFLAGS := -std=c++20 -g -O0 -fopenmp
 ↪  -mmacosx-version-min=14.0

# Define external includes
EIGEN = $(HOME)/opt/eigen-3.4.0/
NLOPTINC = $(HOME)/opt/nlopt-2.7.1/include
SELF = ./include
USR = /opt/homebrew/include
OMPINC = $(HOME)/opt/openmp/include
SVMINC = $(HOME)/opt/libsvm

# Define external libs
NLOPTLIB = $(HOME)/opt/nlopt-2.7.1/lib
OMPLIB = $(HOME)/opt/openmp/lib
SVMLIB = $(HOME)/opt/libsvm

```

The first variable should be modified to reflect the C++ compiler installed in your system. The second one contains the compilation flags which can be modified according to your needs. Right now CMP can be compiled both with debug flags and without, that's why you see two different CXXFLAGS variables. The next lines contain the path of the header and lib folders of the dependencies we installed in the previous sections. Make sure to modify them according to your configuration.

Finally, to compile CMP++ as a static library, simply access the CMP++ folder

via the terminal and type *make release* to compile it without debug flags, *make debug* or *make* to compile both versions.

If the compilation was successful, the lib folder should contain two files **libcmp.a** and **libcmp.dylib** which are the static and dynamic library respectively. If the debug version was compiled, the lib folder should also contain **libcmp_debug.a** and **libcmp_debug.dylib**.

This step should also generate the Doxygen documentation for the code in a folder called docs. To link against CMP++ in your code you should include the header files and link against the library. This can be done by adding the following lines to a makefile

```
# Define CMP++ includes
CMPINC = $(HOME)/opt/CMP++/include
CMPLIB = $(HOME)/opt/CMP++/lib
# Linking against CMP++ static library
LDFLAGS = -lcmp
```

Make sure to replace HOME/opt/CMP++ with the path where you have copied CMP++. See the provided test cases for more details.

3 Test-cases and Verification

This section presents the validation and verification of the CMP++ library using the provided test suite. For each test, we describe the underlying Uncertainty Quantification (UQ) and statistical learning methodology, present the rigorous mathematical formulation, detail the C++ implementation, and analyze the generated plots.

3.1 Probability Distributions & Sliced-Wasserstein Metrics

This test case, implemented in **distribution.cpp**, validates the multivariate probability distribution module and Sliced-Wasserstein distance calculations.

3.1.1 Mathematical Formulation

To draw samples from a Multivariate Normal Distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, CMP++ performs an $\mathbf{LDL}^{\mathrm{T}}$ decomposition (or Cholesky $\mathbf{LL}^{\mathrm{T}}$) of the covariance matrix:

$$\boldsymbol{\Sigma} = \mathbf{L}\mathbf{D}\mathbf{L}^{\mathrm{T}} \, , \tag{1}$$

where $\mathbf{L}$ is a lower unit triangular matrix and $\mathbf{D}$ is a diagonal matrix. A sample is then drawn by generating $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and computing:

$$\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{D}^{1/2}\mathbf{z} \, . \tag{2}$$

To verify that the generated empirical distribution converges to the target distribution, we compute the Sliced-Wasserstein distance. The p-Wasserstein distance between two probability measures μ and ν in $\mathbb{R}^{\mathrm{d}}$ is defined via the optimal transport problem:

$$\mathrm{W}_{\mathrm{p}}(\mu, \nu) = \left(\inf_{\gamma \in \Pi(\mu,\nu)} \int_{\mathbb{R}^{\mathrm{d}} \times \mathbb{R}^{\mathrm{d}}} \|\mathbf{x} - \mathbf{y}\|^{\mathrm{p}} \, \mathrm{d}\gamma(\mathbf{x}, \mathbf{y}) \right)^{1/\mathrm{p}} \, . \tag{3}$$

Because optimal transport is computationally expensive in high dimensions, we project the high-dimensional distributions onto 1D lines. The Sliced-Wasserstein distance SW_{p} is defined by integrating the 1D Wasserstein distances over the unit hypersphere $\mathrm{S}^{\mathrm{d}-1}$:

$$\mathrm{SW}_{\mathrm{p}}(\mu, \nu) = \left(\frac{1}{|\mathrm{S}^{\mathrm{d}-1}|} \int_{\mathrm{S}^{\mathrm{d}-1}} \mathrm{W}_{\mathrm{p}}^{\mathrm{p}}(\theta_{\#}\mu, \theta_{\#}\nu) \, \mathrm{d}\theta \right)^{1/\mathrm{p}} \, , \tag{4}$$

where $\theta_{\#}\mu$ denotes the push-forward of μ under the projection operator $\theta(\mathbf{x}) = \langle \mathbf{x}, \boldsymbol{\theta} \rangle$. Empirically, for a set of L random projection angles $\{\boldsymbol{\theta}_{\mathrm{l}}\}_{\mathrm{l}=1}^{\mathrm{L}}$, the distance is approximated as:

$$\mathrm{SW}_{\mathrm{p}}^{\mathrm{p}}(\mu, \nu) \approx \frac{1}{\mathrm{L}} \sum_{\mathrm{l}=1}^{\mathrm{L}} \mathrm{W}_{\mathrm{p}}^{\mathrm{p}} \left(\{\langle \mathbf{x}_{\mathrm{i}}, \boldsymbol{\theta}_{\mathrm{l}} \rangle\}_{\mathrm{i}=1}^{\mathrm{N}}, \{\langle \mathbf{y}_{\mathrm{j}}, \boldsymbol{\theta}_{\mathrm{l}} \rangle\}_{\mathrm{j}=1}^{\mathrm{N}} \right) \, , \tag{5}$$

where the 1D Wasserstein distance between the projected sorted samples is computed simply as $\frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} |\mathrm{x}_{(\mathrm{i})} - \mathrm{y}_{(\mathrm{i})}|^{\mathrm{p}}$.

3.1.2 C++ Implementation & Verification

In this test, a 2D Multivariate Normal distribution is defined with:

$$\boldsymbol{\mu} = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} 1.0 & 0.8 \\ 0.8 & 1.0 \end{bmatrix} \, . \tag{6}$$

We draw 10,000 samples and plot them to verify correlation.

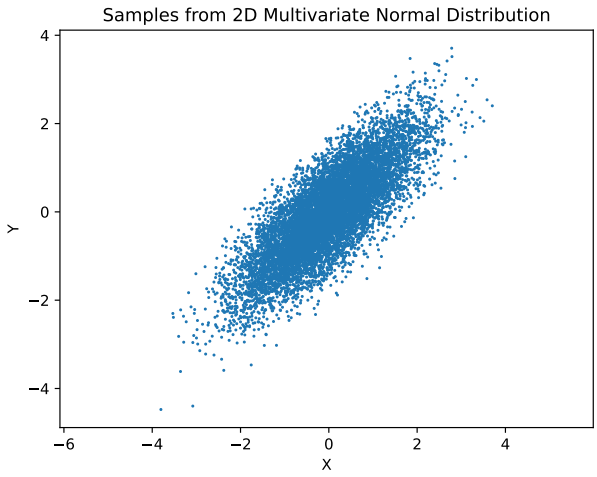


Figure 1: Samples drawn from the 2D Multivariate Normal distribution, showcasing the strong positive correlation ($\rho = 0.8$).

The computed Sliced-Wasserstein distance ($\mathrm{p} = 2,100$ slices) between two independent batches of $5,000$ samples is approximately 1.3×10^{-4}, confirming that the sampler generates statistically indistinguishable realizations from the target density.

3.2 Markov Chain Monte Carlo (MCMC) Diagnostics

The test case **mcmc.cpp** samples a highly bimodal posterior distribution:

$$\log \mathrm{p}(\mathrm{x}) \propto -10 \left(\mathrm{x}^2 - 1\right)^2 \ , \tag{7}$$

which has modes at $\mathrm{x} = \pm 1$.

3.2.1 Mathematical Formulation

At each step of the Metropolis-Hastings (MH) algorithm, a candidate y is proposed from $\mathrm{q}(\mathrm{y}|\mathrm{x})$ and accepted with probability:

$$\alpha(\mathrm{x}, \mathrm{y}) = \min\left(1, \frac{\mathrm{p}(\mathrm{y})\mathrm{q}(\mathrm{x}|\mathrm{y})}{\mathrm{p}(\mathrm{x})\mathrm{q}(\mathrm{y}|\mathrm{x})}\right) \ . \tag{8}$$

In Delayed Rejection (DR), if a proposal y_1 is rejected, a second-stage proposal y_2 is generated from $\mathrm{q}_2(\mathrm{y}_2|\mathrm{x}, \mathrm{y}_1)$. The second-stage acceptance probability is:

$$\alpha_2(\mathrm{x}, \mathrm{y}_1, \mathrm{y}_2) = \min\left(1, \frac{\mathrm{p}(\mathrm{y}_2)\mathrm{q}_1(\mathrm{y}_1|\mathrm{y}_2)\mathrm{q}_2(\mathrm{x}|\mathrm{y}_1, \mathrm{y}_2)[1 - \alpha_1(\mathrm{y}_2, \mathrm{y}_1)]}{\mathrm{p}(\mathrm{x})\mathrm{q}_1(\mathrm{y}_1|\mathrm{x})\mathrm{q}_2(\mathrm{y}_2|\mathrm{x}, \mathrm{y}_1)[1 - \alpha_1(\mathrm{x}, \mathrm{y}_1)]}\right) . \quad (9)$$

In Adaptive Metropolis (AM), the proposal covariance $\mathbf{\Sigma}_\mathrm{t}$ is updated based on the history of the chain:

$$\mathbf{\Sigma}_\mathrm{t} = \mathrm{s}_\mathrm{d}\,\mathrm{Cov}(\mathrm{x}_0, \ldots, \mathrm{x}_{\mathrm{t}-1}) + \mathrm{s}_\mathrm{d}\varepsilon\mathbf{I}_\mathrm{d} , \quad (10)$$

where $\mathrm{s}_\mathrm{d} = 2.38^2/\mathrm{d}$ is the optimal scaling factor and ε is a small nugget for stability.

The autocorrelation function of a chain $\{\mathrm{x}_\mathrm{t}\}_{\mathrm{t}=1}^{\mathrm{N}}$ at lag k is computed using FFT for speed:

$$\rho(\mathrm{k}) = \frac{\mathbb{E}[(\mathrm{x}_\mathrm{t} - \mu)(\mathrm{x}_{\mathrm{t}+\mathrm{k}} - \mu)]}{\sigma^2} . \quad (11)$$

The Effective Sample Size (ESS) is defined as:

$$\mathrm{ESS} = \frac{\mathrm{N}}{1 + 2\sum_{\mathrm{k}=1}^{\infty}\rho(\mathrm{k})} . \quad (12)$$

3.2.2 C++ Implementation & Verification

We run the DRAM sampler, discard the first $10,000$ samples as burn-in, draw $1,000,000$ samples, and thin the chain by a factor of 10.

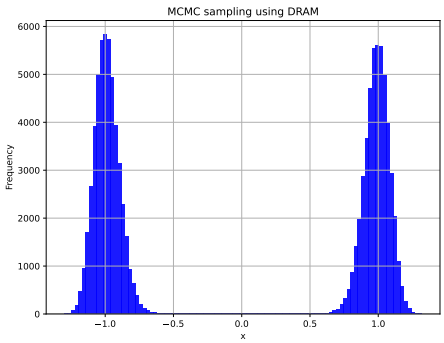


Figure 2: Empirical histogram of MCMC samples showing successful capturing of the bimodal distribution.

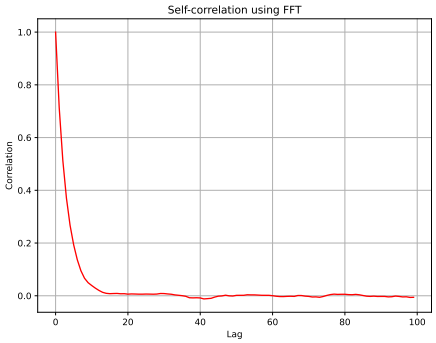


Figure 3: Autocorrelation function of the chain computed efficiently via FFT.

The diagnostic output yields a self-correlation length of approximately 3.7 steps and an ESS of over $15,500$, proving high sampling efficiency and low correlation between samples.

3.3 Kernel Density Estimation (KDE)

Implemented in **kde.cpp**, this test demonstrates non-parametric density estimation.

3.3.1 Mathematical Formulation

For a dataset $\{\mathbf{x}_i\}_{i=1}^{N}$ in $\mathbb{R}^d$, the Kernel Density Estimator is:

$$\hat{f}_{\mathbf{H}}(\mathbf{x}) = \frac{1}{N}\sum_{i=1}^{N} K_{\mathbf{H}}(\mathbf{x} - \mathbf{x}_i) \ , \tag{13}$$

where $K_{\mathbf{H}}(\mathbf{z}) = |\mathbf{H}|^{-1/2} K(\mathbf{H}^{-1/2}\mathbf{z})$, with $\mathbf{H}$ being the symmetric positive-definite bandwidth matrix. The multivariate Gaussian kernel is:

$$K(\mathbf{z}) = (2\pi)^{-d/2} \exp\left(-\frac{1}{2}\|\mathbf{z}\|^2\right) \ . \tag{14}$$

Silverman's rule of thumb selects a diagonal bandwidth matrix $\mathbf{H}_{jj} = h_j^2$ where:

$$h_j = \left(\frac{4}{d+2}\right)^{\frac{1}{d+4}} N^{-\frac{1}{d+4}} \sigma_j \ , \tag{15}$$

with σ_j being the standard deviation of the j-th coordinate. The bandwidth matrix is optimized by maximizing the Leave-One-Out Likelihood Cross-Validation (LOOCV) score:

$$\text{LOOCV}(\mathbf{H}) = \sum_{i=1}^{N} \log \hat{f}_{\mathbf{H},-i}(\mathbf{x}_i) \ , \tag{16}$$

where $\hat{f}_{\mathbf{H},-i}$ is the KDE constructed omitting the i-th observation.

3.3.2 C++ Implementation & Verification

We generate $1,000$ samples from a Gaussian Mixture Model (GMM) with weights 0.3 and 0.7:

$$p(\mathbf{x}) = 0.3 \cdot \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + 0.7 \cdot \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2) \ , \tag{17}$$

where $\boldsymbol{\mu}_1 = [-0.5, 0.5]^{\mathrm{T}}$ and $\boldsymbol{\mu}_2 = [0.5, -0.5]^{\mathrm{T}}$.

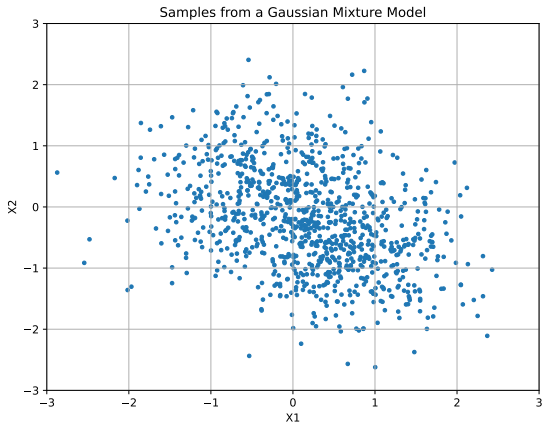


Figure 4: Scatter plot of GMM samples.

The true density at origin is 0.202. The Silverman KDE yields 0.156 with bandwidth:

$$\mathbf{H} = \begin{bmatrix} 0.243 & 0.0 \\ 0.0 & 0.241 \end{bmatrix} . \tag{18}$$

Maximizing the LOOCV score refines the bandwidth to:

$$\mathbf{H}_{\text{opt}} = \begin{bmatrix} 0.065 & -0.008 \\ -0.008 & 0.076 \end{bmatrix} , \tag{19}$$

which brings the estimated density at origin to 0.196, validating the optimization.

3.4 Supervised Classification: KDE, SVM, and Multi-class Decision Boundaries

The file **classifier.cpp** compares KDE and Support Vector Machine (SVM) binary classifiers on the synthetic 2D moons dataset.

3.4.1 Mathematical Formulation

The Support Vector Machine (SVM) binary classifier constructs a decision boundary by finding the hyperplane that maximizes the margin. In kernelized

dual form, we solve:

$$\max_{\boldsymbol{\alpha}} \sum_{\mathrm{i}=1}^{\mathrm{N}} \alpha_{\mathrm{i}} - \frac{1}{2} \sum_{\mathrm{i}=1}^{\mathrm{N}} \sum_{\mathrm{j}=1}^{\mathrm{N}} \alpha_{\mathrm{i}} \alpha_{\mathrm{j}} \mathrm{y}_{\mathrm{i}} \mathrm{y}_{\mathrm{j}} \mathrm{K}(\mathbf{x}_{\mathrm{i}}, \mathbf{x}_{\mathrm{j}}) \ , \tag{20}$$

subject to $0 \leq \alpha_{\mathrm{i}} \leq \mathrm{C}$ and $\sum_{\mathrm{i}=1}^{\mathrm{N}} \alpha_{\mathrm{i}} \mathrm{y}_{\mathrm{i}} = 0$. The Radial Basis Function (RBF) kernel is defined as:

$$\mathrm{K}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\right) \ . \tag{21}$$

Hyperparameter tuning of C and γ is performed using a span-bound estimate to minimize the leave-one-out error.

3.4.2 C++ Implementation & Verification

We compare the classification decision boundaries generated by the optimized KDE and SVM.

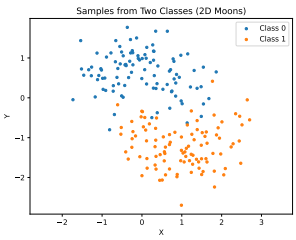


Figure 5: Raw moons dataset.

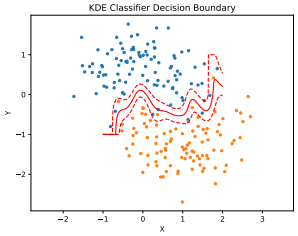


Figure 6: KDE decision boundary.

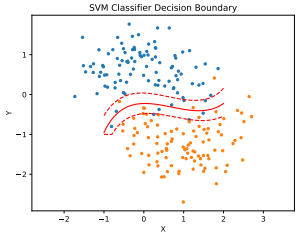


Figure 7: SVM decision boundary.

Additionally, **svm.cpp** performs multi-class SVM classification on a complex 10-class dataset, finding optimized RBF parameters ($\gamma = 0.14$, $\mathrm{C} = 10.14$) and achieving a training accuracy of 98%.

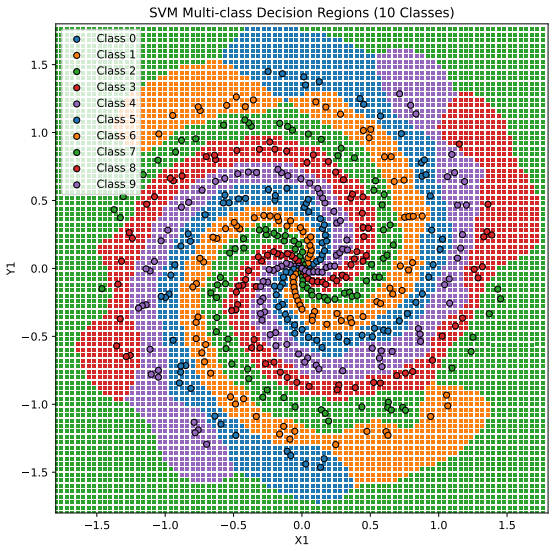


Figure 8: Decision regions for the 10-class SVM classifier.

3.5 Gaussian Process (GP) Regression

Implemented in **gp.cpp**, this test conditions a Gaussian Process model on noisy data generated from $\mathrm{y} = \sin(2\pi \mathrm{x}) + 1 + \varepsilon$.

3.5.1 Mathematical Formulation

A Gaussian Process is a collection of random variables, any finite number of which have a joint Gaussian distribution. It is completely defined by its mean function $\mathrm{m}(\mathbf{x})$ and covariance function $\mathrm{k}(\mathbf{x}, \mathbf{x}')$:

$$\mathrm{f}(\mathbf{x}) \sim \mathcal{GP}\left(\mathrm{m}(\mathbf{x}), \mathrm{k}(\mathbf{x}, \mathbf{x}')\right) \ . \tag{22}$$

Given training observations $\mathbf{y}$ at inputs $\mathbf{X}$, the joint distribution of training outputs and prediction outputs $\mathbf{f}_*$ at test points $\mathbf{X}_*$ is:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mathrm{m}(\mathbf{X}) \\ \mathrm{m}(\mathbf{X}_*) \end{bmatrix}, \begin{bmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_{\mathrm{n}}^2 \mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{bmatrix} \right) \ . \tag{23}$$

Conditioning on $\mathbf{y}$ yields the posterior distribution $\mathbf{f}_* | \mathbf{X}, \mathbf{y}, \mathbf{X}_* \sim \mathcal{N}(\bar{\mathbf{f}}_*, \mathrm{cov}(\mathbf{f}_*))$:

$$\bar{\mathbf{f}}_* = \mathrm{m}(\mathbf{X}_*) + \mathbf{K}(\mathbf{X}_*, \mathbf{X}) \left[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_{\mathrm{n}}^2 \mathbf{I}\right]^{-1} (\mathbf{y} - \mathrm{m}(\mathbf{X})) \ , \tag{24}$$

$$\operatorname{cov}(\mathbf{f}_*) = \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}(\mathbf{X}_*, \mathbf{X}) \left[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_{\mathrm{n}}^2 \mathbf{I}\right]^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \,. \tag{25}$$

We optimize the hyperparameters $\boldsymbol{\theta} = (\sigma_{\mathrm{f}}, \ell, \sigma_{\mathrm{n}})$ of the Squared Exponential kernel by minimizing the Leave-One-Out (LOO) cross-validation Mean Squared Error:

$$\mathrm{MSE}_{\mathrm{LOO}} = \frac{1}{\mathrm{N}} \sum_{\mathrm{i}=1}^{\mathrm{N}} \left(\mathrm{y}_{\mathrm{i}} - \hat{\mu}_{-\mathrm{i}}(\mathrm{x}_{\mathrm{i}})\right)^2 \,, \tag{26}$$

where $\hat{\mu}_{-\mathrm{i}}(\mathrm{x}_{\mathrm{i}})$ is the GP prediction at x_{i} computed by omitting the i-th observation.

3.5.2 C++ Implementation & Verification

We compare the prior GP, the conditioned posterior GP, and the optimized posterior GP.

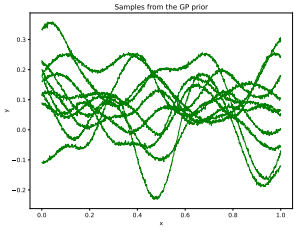


Figure 9: Samples from GP prior.

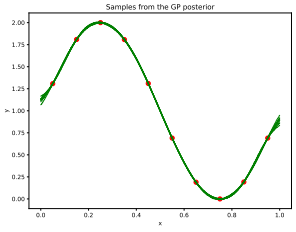


Figure 10: Samples from posterior.

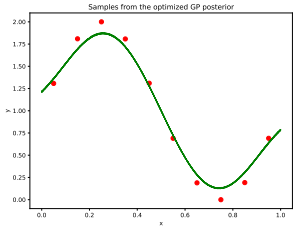


Figure 11: Samples from optimized posterior.

The cross-validation MSE drops from 0.302 (initial hyperparameters) to 0.040 after optimization, demonstrating the importance of hyperparameter optimization.

3.6 Multi-Output GP with Dimensionality Reduction

The test case **multi_gp.cpp** fits a multi-output GP using Principal Component Analysis (PCA) to project a highly correlated 3D output vector into a 2D latent space.

3.6.1 Mathematical Formulation

For a multi-output training matrix $\mathbf{Y} \in \mathbb{R}^{\mathrm{N} \times \mathrm{D}}$ (where $\mathrm{D} = 3$ is high-dimensional), we compute the singular value decomposition (SVD) of the centered matrix

to obtain the projection matrix $\mathbf{W} \in \mathbb{R}^{\mathrm{D}\times\mathrm{r}}$ corresponding to the top $\mathrm{r} = 2$ principal components:

$$\mathbf{Z} = (\mathbf{Y} - \mathbf{1}\boldsymbol{\mu}^{\mathrm{T}})\mathbf{W} \ , \tag{27}$$

where $\mathbf{Z} \in \mathbb{R}^{\mathrm{N}\times\mathrm{r}}$ is the matrix of low-dimensional latent coordinates. We fit r independent Gaussian Processes to each column of $\mathbf{Z}$. Predictions at test inputs are projected back to the physical space:

$$\hat{\mathbf{Y}}_* = \hat{\mathbf{Z}}_* \mathbf{W}^{\mathrm{T}} + \mathbf{1}\boldsymbol{\mu}^{\mathrm{T}} \ , \tag{28}$$

with the prediction covariance similarly mapped.

3.6.2 C++ Implementation & Verification

We fit the multi-output GP and verify the reconstructions against observations.

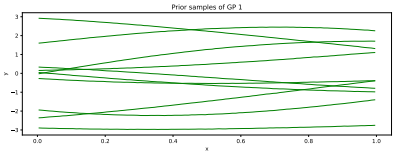


Figure 12: GP prior samples for the latent dimensions.

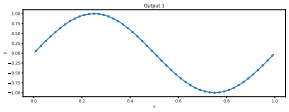


Figure 13: Output 1 regression.

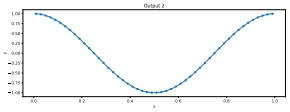


Figure 14: Output 2 regression.

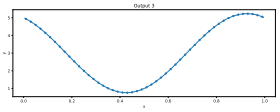


Figure 15: Output 3 regression.

Figure 16: Multi-output predictions reconstructed from the PCA projection, matching observations very closely.

3.7 Polynomial Chaos Expansion (PCE)

The test case **poly.cpp** fits a 1D Legendre-polynomial expansion to approximate a slice of the Ishigami function:

$$\mathrm{f}(\mathrm{x}_1, 0, 0) = \sin(\mathrm{x}_1) \ . \tag{29}$$

3.7.1 Mathematical Formulation

A spectral Polynomial Chaos Expansion (PCE) represents a model output $\mathrm{Y} = \mathrm{f}(\boldsymbol{\xi})$ as a series of orthogonal polynomials:

$$\mathrm{f}(\boldsymbol{\xi}) = \sum_{\boldsymbol{\alpha} \in \mathcal{A}} \mathrm{c}_{\boldsymbol{\alpha}} \Psi_{\boldsymbol{\alpha}}(\boldsymbol{\xi}) \ , \tag{30}$$

where $\boldsymbol{\alpha}$ is a multi-index, and $\Psi_{\boldsymbol{\alpha}}$ are multivariate orthogonal polynomials satisfying:

$$\langle \Psi_{\boldsymbol{\alpha}}, \Psi_{\boldsymbol{\beta}} \rangle = \int \Psi_{\boldsymbol{\alpha}}(\boldsymbol{\xi}) \Psi_{\boldsymbol{\beta}}(\boldsymbol{\xi}) \rho(\boldsymbol{\xi}) \, \mathrm{d}\boldsymbol{\xi} = \gamma_{\boldsymbol{\alpha}} \delta_{\boldsymbol{\alpha}\boldsymbol{\beta}} \ . \tag{31}$$

For Legendre polynomials corresponding to uniform distributions on $[-1, 1]$, the coefficients $\mathrm{c}_{\boldsymbol{\alpha}}$ are computed via numerical integration (Gauss-Legendre quadrature):

$$\mathrm{c}_{\boldsymbol{\alpha}} = \frac{1}{\gamma_{\boldsymbol{\alpha}}} \sum_{\mathrm{i}=1}^{\mathrm{M}} \mathrm{w}_{\mathrm{i}} \mathrm{f}(\boldsymbol{\xi}_{\mathrm{i}}) \Psi_{\boldsymbol{\alpha}}(\boldsymbol{\xi}_{\mathrm{i}}) \ . \tag{32}$$

3.7.2 C++ Implementation & Verification

We fit a Legendre PCE to the Ishigami slice and compare the Sobol sensitivity index.

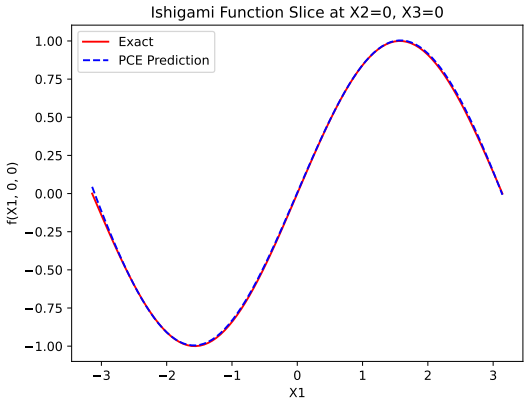


Figure 17: Legendre PCE approximation compared with the exact Ishigami slice.

The analytical Sobol index S_1 is 0.818073, whereas the PCE yields 0.817885 (absolute error of 0.000188), validating the accuracy of the spectral projections.

3.8 Global Sensitivity Analysis: Sobol Indices

In **sobol.cpp**, we compute the Sobol sensitivity indices of the 3D Ishigami function:

$$\mathrm{f}(\mathrm{x}_1, \mathrm{x}_2, \mathrm{x}_3) = \sin(\mathrm{x}_1) + \mathrm{a}\sin^2(\mathrm{x}_2) + \mathrm{b}\mathrm{x}_3^4 \sin(\mathrm{x}_1) \ , \tag{33}$$

with $\mathrm{a} = 7, \mathrm{b} = 0.1$.

3.8.1 Mathematical Formulation

The ANOVA decomposition decomposes the total variance of the model output Y into contributions from individual inputs and their interactions:

$$\mathrm{Var}(\mathrm{Y}) = \sum_{\mathrm{i}=1}^{\mathrm{d}} \mathrm{V}_\mathrm{i} + \sum_{1 \le \mathrm{i} < \mathrm{j} \le \mathrm{d}} \mathrm{V}_{\mathrm{ij}} + \cdots + \mathrm{V}_{1\ldots\mathrm{d}} \ , \tag{34}$$

where $\mathrm{V}_\mathrm{i} = \mathrm{Var}_{\mathrm{X}_\mathrm{i}}\left(\mathbb{E}_{\mathrm{X}_{\sim \mathrm{i}}}[\mathrm{Y}|\mathrm{X}_\mathrm{i}]\right)$. The first-order and total sensitivity indices are defined as:

$$\mathrm{S}_\mathrm{i} = \frac{\mathrm{V}_\mathrm{i}}{\mathrm{Var}(\mathrm{Y})}, \quad \mathrm{S}_{\mathrm{T}_\mathrm{i}} = 1 - \frac{\mathrm{Var}_{\mathrm{X}_{\sim \mathrm{i}}}\left(\mathbb{E}_{\mathrm{X}_\mathrm{i}}[\mathrm{Y}|\mathrm{X}_{\sim \mathrm{i}}]\right)}{\mathrm{Var}(\mathrm{Y})} \ . \tag{35}$$

Using Saltelli sampling, we construct two matrices $\mathbf{A}$ and $\mathbf{B}$, and define a matrix $\mathbf{C}_\mathrm{i}$ where the i-th column is taken from $\mathbf{B}$ and the rest from $\mathbf{A}$. The estimator for the first-order index is:

$$\mathrm{V}_\mathrm{i} \approx \frac{1}{\mathrm{N}} \sum_{\mathrm{j}=1}^{\mathrm{N}} \mathrm{f}(\mathbf{B})_\mathrm{j} \left(\mathrm{f}(\mathbf{C}_\mathrm{i})_\mathrm{j} - \mathrm{f}(\mathbf{A})_\mathrm{j}\right) \ . \tag{36}$$

3.8.2 C++ Implementation & Verification

We generate Saltelli sample grids, compute Sobol indices, and construct bootstrap confidence intervals.

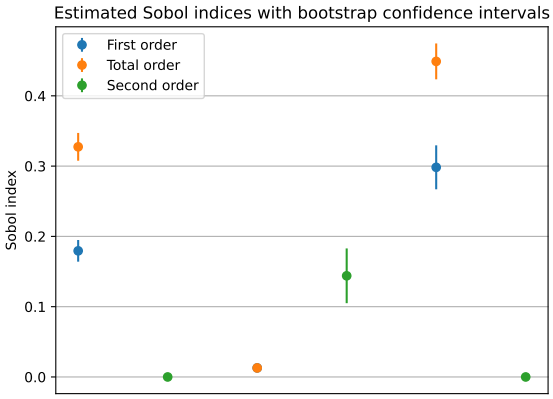


Figure 18: First-order, second-order, and total sensitivity indices computed using Saltelli sampling with 30 bootstrap resamples.

The results verify the theoretical values (e.g. total order index for $\mathrm{X}_3 \approx 0.449$, matching the analytical 0.430).

3.9 Evolutionary MCMC (EMCMC)

In **emcmc.cpp**, multiple chains interact simultaneously to explore a highly multimodal score function:

$$\log \mathrm{p}(\mathbf{x}) \propto -8 \left(\|\mathbf{x}\|^2 - 1 \right)^2 \ . \tag{37}$$

3.9.1 Mathematical Formulation

Evolutionary MCMC combines Markov Chain Monte Carlo with Genetic Algorithms. Multiple chains are run in parallel, and joint proposals are generated using a differential evolution scheme. For chain i, the candidate state is proposed as:

$$\mathbf{x}_\mathrm{i}^* = \mathbf{x}_\mathrm{i} + \gamma(\mathbf{x}_{\mathrm{r}_1} - \mathbf{x}_{\mathrm{r}_2}) + \mathbf{e} \ , \tag{38}$$

where $\mathrm{r}_1, \mathrm{r}_2$ are two randomly selected distinct chains, and $\mathbf{e} \sim \mathcal{N}(\mathbf{0}, \delta\mathbf{I})$ is a small random perturbation. The optimal scaling factor is $\gamma = 2.38/\sqrt{2\mathrm{d}}$, and every 10th step we set $\gamma = 1.0$ to enable mode-jumping.

3.9.2 C++ Implementation & Verification

We run 10 parallel chains for $100,000$ iterations and plot the histogram of the first chain.

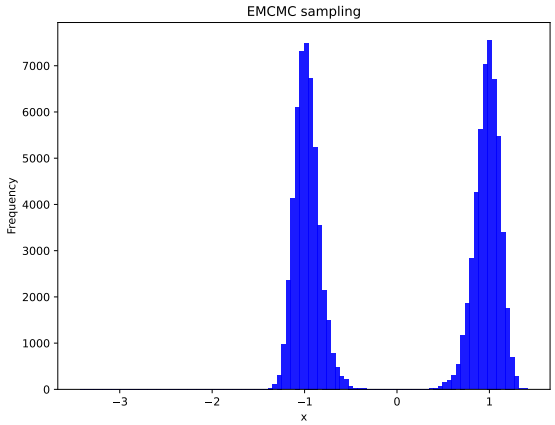


Figure 19: Histogram of samples drawn using Evolutionary MCMC, demonstrating robust mode-crossing and uniform exploration of the target space.

3.10 Dirichlet Process Mixture Models (DPMM)

The test case **cluster.cpp** performs clustering using a Dirichlet Process Mixture Model (DPMM).

3.10.1 Mathematical Formulation

The Dirichlet Process Mixture Model (DPMM) is a Bayesian non-parametric method that infers the number of mixture components from the data. The model is formulated as:

$$\mathbf{x}_\mathrm{i}|\mathrm{c}_\mathrm{i}, \boldsymbol{\theta} \sim \mathrm{F}(\boldsymbol{\theta}_{\mathrm{c}_\mathrm{i}}) \tag{39}$$

$$\mathrm{c}_\mathrm{i}|\mathbf{p} \sim \mathrm{Categorical}(\mathbf{p}) \tag{40}$$

$$\mathbf{p} \sim \mathrm{GEM}(\alpha)\ , \tag{41}$$

where α is the concentration parameter. Using a Gibbs sampler under the Chinese Restaurant Process representation, the probability of assigning observation i to an existing cluster k with $\mathrm{n}_{-\mathrm{i},\mathrm{k}}$ members is:

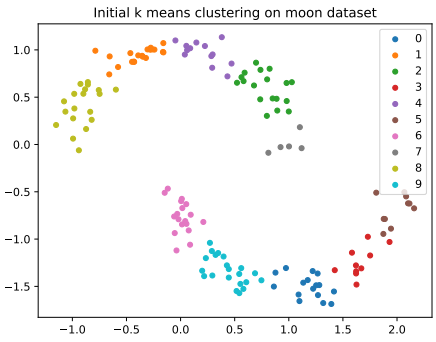
$$\mathrm{P}(\mathrm{c}_\mathrm{i} = \mathrm{k}|\mathbf{c}_{-\mathrm{i}}, \mathbf{x}_\mathrm{i}) \propto \frac{\mathrm{n}_{-\mathrm{i},\mathrm{k}}}{\mathrm{N} - 1 + \alpha}\mathrm{p}(\mathbf{x}_\mathrm{i}|\boldsymbol{\theta}_\mathrm{k})\ , \tag{42}$$

and to a new cluster:

$$\mathrm{P}(\mathrm{c_i} = \mathrm{k_{new}} | \mathbf{c}_{-\mathrm{i}}, \mathbf{x_i}) \propto \frac{\alpha}{\mathrm{N} - 1 + \alpha} \int \mathrm{p}(\mathbf{x_i}|\boldsymbol{\theta})\, \mathrm{dH}(\boldsymbol{\theta}) \; . \tag{43}$$

3.10.2 C++ Implementation & Verification

We cluster synthetic data and compare the DPMM boundaries against K-means.



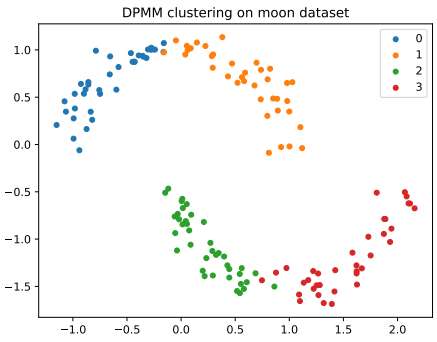


Figure 20: Initial K-means clustering. Figure 21: Refined DPMM clustering.

Figure 22: Clustering progression showing refinement of boundaries and dynamic class count inference using DPMM.

3.11 Bayesian Calibration with Model Discrepancy

Implemented in **calibration.cpp**, a physical model is calibrated under the Kennedy-O'Hagan (KOH) framework.

3.11.1 Mathematical Formulation

The Kennedy-O'Hagan framework models structural model discrepancy using a Gaussian Process:

$$\mathrm{y_{obs}}(\mathbf{x}) = \eta(\mathbf{x}, \boldsymbol{\theta}) + \delta(\mathbf{x}) + \varepsilon \; , \tag{44}$$

where $\eta(\mathbf{x}, \boldsymbol{\theta}) = \mathrm{c}\mathbf{x}^2$ is the computer model scale parameter, $\delta(\mathbf{x}) \sim \mathcal{GP}(0, \mathrm{k}_\delta(\mathbf{x}, \mathbf{x}'))$ is the discrepancy term modeled by a GP with hyperparameters $\boldsymbol{\psi} = (\sigma_\mathrm{f}, \ell)$,

and $\varepsilon \sim \mathcal{N}(0, \sigma_{\mathrm{n}}^2)$ is the noise. The joint likelihood of the parameter c and the GP hyperparameters $\boldsymbol{\psi}$ is:

$$\mathbf{y}_{\mathrm{obs}} \sim \mathcal{N}\left(\boldsymbol{\eta}(\mathbf{X}, \mathrm{c}), \mathbf{K}_{\delta}(\mathbf{X}, \mathbf{X}) + \sigma_{\mathrm{n}}^2 \mathbf{I}\right) \ . \tag{45}$$

We specify prior distributions for c and $\boldsymbol{\psi}$ and sample the joint posterior using the Metropolis-Hastings MCMC sampler.

3.11.2 C++ Implementation & Verification

We fit the parameters and display posterior marginals and the final discrepancy adjustment fit.

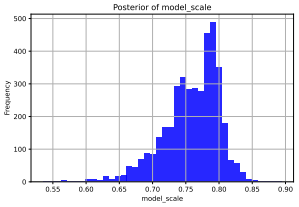


Figure 23: Posterior of c.

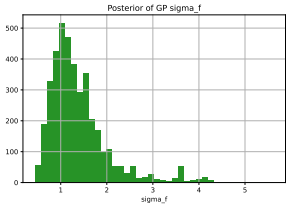


Figure 24: Posterior of GP σ_{f}.

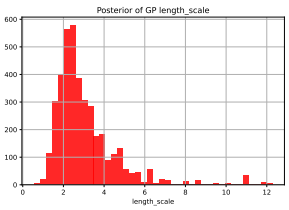


Figure 25: Posterior of GP ℓ.

Figure 26: Marginal posterior histograms for model scale and discrepancy hyperparameters.

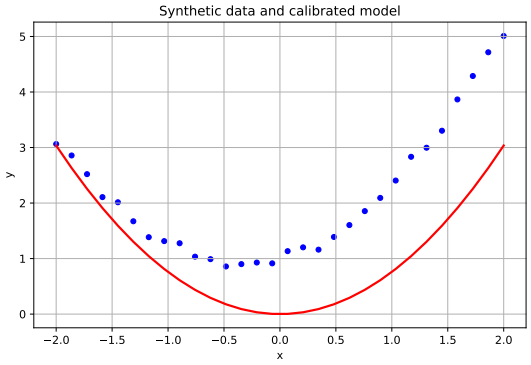


Figure 27: Observations, exact model prediction, and calibrated model including GP discrepancy adjustment.

The calibration yields an estimated model scale $\mathrm{c} \approx 0.759 \pm 0.042$, successfully capturing the underlying parameter despite structural model discrepancies.